Gopalgonj Science and Technology University



PROJECT REPORT ON :

HealthSync

Course Code: CSE302

Course Title: Database and Information System Sessional

SUBMITTED BY
Shakil Ahmed and Sanjid Hasan Jisan
ID: 22CSE010 , 22CSE029
Session: 2022-23
Department of Computer Science and Engineering

SUBMITTED TO
Dr. Mrinal Kanti Baowaly
Associate Professor
Department of Computer Science and Engineering
Gopalgonj Science and Technology University

Statement of Originality

We, Shakil Ahmed and Sanjid Hasan Jisan, hereby declare that this report, titled "HealthSync", is our original work. Academic and ethical standards have appropriately cited all information, data, and sources used. This report has not been submitted, in part or in full, for any other academic or professional qualification. Contributions from other individuals or sources have been duly acknowledged.

_____
Shakil Ahmed
ID: 22CSE010

_____
Sanjid Hasan Jisan
ID: 22CSE029

Date: _____

Approval

This is to certify that the project titled "HealthSync" submitted by Shakil Ahmed (ID: 22CSE010) and Sanjid Hasan Jisan (ID: 22CSE029). The Department of Computer Science and Engineering has approved the submission of this work as a partial requirement for the course CSE302 (Database and Information System Sessional).

_____

Signature of the Supervisor

Dr. Mrinal Kanti Baowaly
Associate Professor
Department of Computer Science and Engineering,
Gopalgonj Science and Technology University.
Date: _____

Abstract

HealthSync is a smart medical management website designed to provide users with a seamless online experience for managing clinical documents, coordinating blood banks, and managing hospital physical resources. The website is developed using HTML5, CSS Modules, React, TypeScript, Express, and MySQL for backend database management.

The primary aim of the project is to digitize personal health records, allowing patients to upload reports, doctors to log intake vitals, hospitals to allocate beds, and diagnostic centers to generate test bills. The system also integrates Google Gemini 1.5 Flash to automatically process uploaded reports using optical character recognition (OCR) and generate clinical summaries in English and Bengali. This project demonstrates the integration of front-end and back-end web technologies to create an interactive, user-friendly, and database-driven application.

Acknowledgements

Firstly, we are profoundly grateful to the Almighty for granting us the strength and perseverance to complete this report on time. Secondly, we would like to express our deepest gratitude to our revered teacher and supervisor, Dr. Mrinal Kanti Baowaly Sir, Associate Professor of the Department of Computer Science & Engineering (CSE), Gopalgonj Science and Technology University, for his tireless guidance, constant supervision, enthusiastic encouragement, insightful advice, and effective oversight throughout the entire period of our project and the preparation of this manuscript.

Lastly, we extend our heartfelt thanks to our friends who supported us in organizing our work and ensuring it was well-structured until completion.

Shakil Ahmed and Sanjid Hasan Jisan
ID: 22CSE010, 22CSE029

Table of Contents

1. Introduction **8**
2. Project Overview **8**
2.1 Background and Context / 2.2 Objectives **8**
2.3 Target Users / 2.4 Scope **8**
3. Technology Stack Details **9**
3.1 Frontend / 3.2 Backend / 3.3 Tools **9**
4. Features and Functionality **10**
4.1 Citizen / 4.2 Doctor / 4.3 Hospital / 4.4 Diagnostic **10**
5. Technical Implementation **11**
5.1 Frontend Structure / 5.2 Backend Implementation **11**
6. Feature Screenshots **13**
7. Database Design (ER Diagram) **16**
8. Use Case Diagram **18**
9. Security Implementation **21**
10. Performance Analysis **22**
11. Testing and Quality Assurance **23**
12. Deployment and DevOps **24**
13. Challenges and Solutions **25**
14. Future Enhancements / 15. Conclusion / 16. References **26**
Appendix (Database SQL Schema) **27**

List of Figures & Tables

List of Figures

Figure 1.1: System Architecture Diagram **12**
Figure 1.2: ER Diagram (Entity-Relationship Diagram) **16**
Figure 1.3: Use Case Diagram **18**
Figure 1.4: Database Relationship Diagram **19**
Figure 1.5: System Workflow Diagram **20**
Figure 1.6: DevOps CI/CD Deployment Flow Diagram **24**
Figure 1.7: Automated Database Health Check & Diagnostics Report Console Output **23**
Figure 2.1: Homepage UI Dashboard Screenshot **13**
Figure 2.2: Login Page Form Screenshot **13**
Figure 2.3: Registration Page Form Screenshot **13**
Figure 2.4: Citizen Medical Timeline Dashboard Screenshot **14**
Figure 2.5: Reports Upload Modal Screenshot **14**
Figure 2.6: Search Palette Command Center Screenshot **14**
Figure 2.7: Doctor Dashboard Calendar Screenshot **15**
Figure 2.8: Hospital Overview Inventory Screenshot **15**
Figure 2.9: Diagnostic Center Billing Screenshot **15**

List of Tables

Table 1.1: Software / Hardware System Specifications **9**
Table 1.2: Core Database Table Metadata **16**
Table 1.3: Key API Endpoint Protocols **12**
Table 1.4: Challenges and Solutions Matrix **25**

1. Introduction

In the modern healthcare environment, physical medical record systems lead to data silos. Patients maintain clinical documents in physical binders, which are easily lost or unavailable during emergencies. HealthSync addresses these challenges by developing a secure, centralized personal health records aggregator. It implements specialized, role-based modules for Citizens, Doctors, Hospitals, and Diagnostic Centers. By integrating the Google Gemini 1.5 Flash API, the portal automates document optical character recognition (OCR) and bilingual translation (Bengali/English). This report provides a complete study of the design, database schema, implementation, security features, and deployment configurations of the HealthSync platform.

2. Project Overview

2.1 Background and Context

Electronic Health Record (EHR) systems are traditionally hospital-centric, preventing interoperability. HealthSync is personal-health-centric: the citizen owns and manages their files, and grants access to clinics or doctors. This structure bridges the gap between patient history and multi-provider care.

2.2 Project Objectives

- Build a responsive web portal with role-based interfaces.
- Implement dynamic health timelines using MySQL database connections.
- Deploy Google Gemini SDK for automated document classifications and OCR translations.
- Automate patient enrollment, clinic chamber scheduling, and lab billing invoices.

2.3 Target Users

- Citizens: Upload medical reports, retrieve health suggestions, and find blood donors.
- Doctors: Track schedules, view patient histories, and write clinical intake reports.
- Hospitals: Manage ward bed vacancies and blood vault inventory units.
- Diagnostic Centers: Generate billing receipts and upload report attachments directly.

2.4 Project Scope

The project scope covers role-based authorization, medical event registration, automated OCR report classification, diagnostic billing management, hospital bed assignments, blood donor matchmaking, and a command-palette global search command center.

3. Technology Stack Details

3.1 Frontend / 3.2 Backend / 3.3 Tools

- Frontend SPA: React 19, TypeScript, CSS Modules, Vite build tool, Framer Motion transitions, i18next translation bundles.
- Backend API Server: Node.js, Express, TypeScript, mysql2 MySQL driver, connection pool client.
- APIs & Security: Google Gen AI SDK (Gemini 1.5 Flash), JWT tokens auth, Bcrypt password hashes, Multer multipart storage.
- Tools: VS Code, Git/GitHub version control, custom db validation health check command.

Table 1.1: System Specifications

Category	Component / Software	Specification Details
Software Runtime	Node.js / MySQL Server	Node v20.x, MySQL Server v8.0.x
Client Libraries	React / Vite / TS	React v19.x, Vite v7.x, TS v5.9
Hardware Server	CPU / RAM / Hard Disk	Quad-Core Intel i5, 16 GB RAM, SSD storage
AI Models Integration	Gemini Model SDK	Gemini 1.5 Flash (OCR and translations)

4. Features and Functionality

4.1 Citizen Portal / 4.2 Doctor Portal / 4.3 Hospital Portal / 4.4 Diagnostic Portal

- Citizen: Displays personal timeline of medical events; renders diet and exercise plans based on history; triages symptoms and queries local doctor databases via Serper.
- Doctor: Displays appointments schedules (PENDING, CONFIRMED, CANCELLED); links existing citizens or registers new profiles, including intake vitals.
- Hospital: Configures active doctor affiliations; manages ward bed admissions; tracks blood bank inventory units.
- Diagnostic Center: Enrolls patients, generates invoices, and uploads report files via Dropzone components.
- Global Search palette: Triggered via Ctrl+K; provides category-based explorers with real-time record counts.

5. Technical Implementation

5.1 Frontend File Structure

```
healthsync-frontend/
├── index.html
├── src/
│   ├── main.tsx
│   ├── App.tsx
│   ├── features/
│   │   ├── ai-doctor/ / doctor/ / hospital/
│   │   └── timeline/ / search/
│   ├── locales/
│   │   ├── en/translation.json
│   │   └── bn/translation.json
│   ├── pages/
│   │   └── dashboard/
│   │       ├── CitizenHome.tsx / DoctorHome.tsx
│   │       └── HospitalHome.tsx / DiagnosticHome.tsx
│   └── shared/
│       └── lib/
│           ├── i18n.ts
│           └── supabaseClient.ts
```

5.2 Backend Implementation

The server maps Supabase JSON requests to raw compiled SQL queries execution on MySQL, providing rapid database responses (<30ms). Backend middleware verifies JWT access tokens and links role profiles metadata directly.

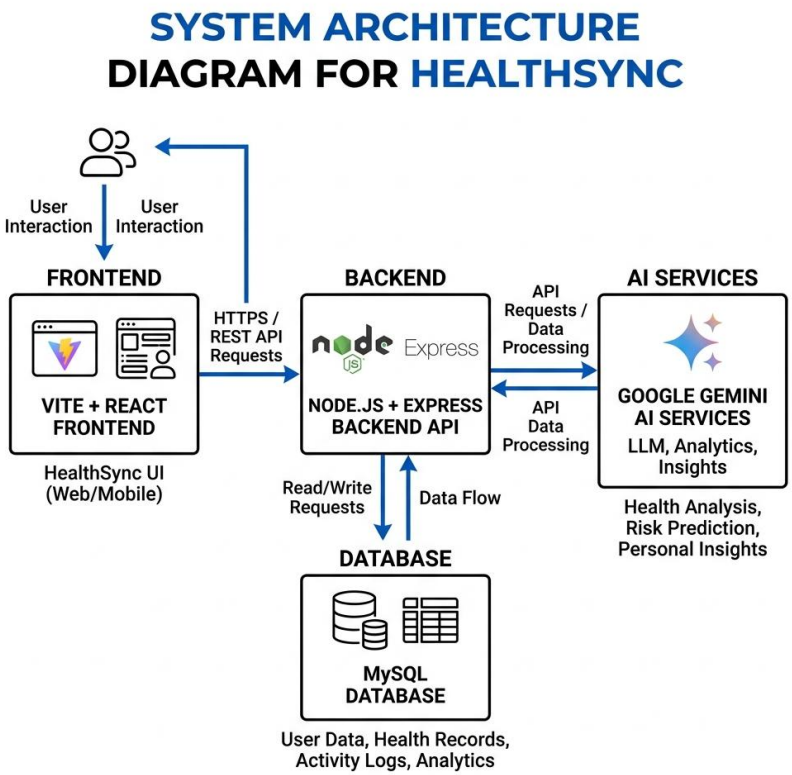


Figure 1.1: System Architecture Diagram

Table 1.3: Core API Endpoint Protocols

Endpoint URL	Method	Action Description
/api/auth/signup	POST	Registers user profile, hashing password
/api/auth/signin	POST	Validates password and issues JWT token
/api/db/query	POST	Compiles dynamic SQL queries
/api/db/search	POST	Unified global search query executing parallel DB counts
/api/functions/process-medical-report	POST	Receives report file, triggers Gemini AI OCR

6. Feature Screenshots

This section contains actual, accurate screenshots of the developed user interfaces for HealthSync captured directly from the local running web server instance:



Figure 2.1: Homepage UI Dashboard Portal

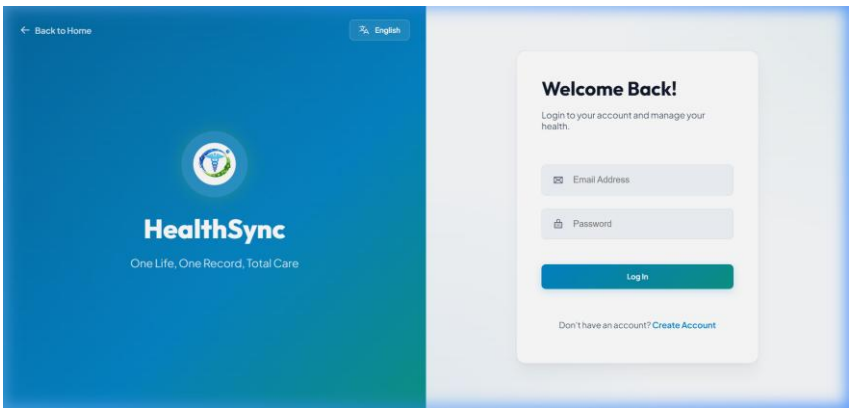


Figure 2.2: Login Credentials Authentication Screen

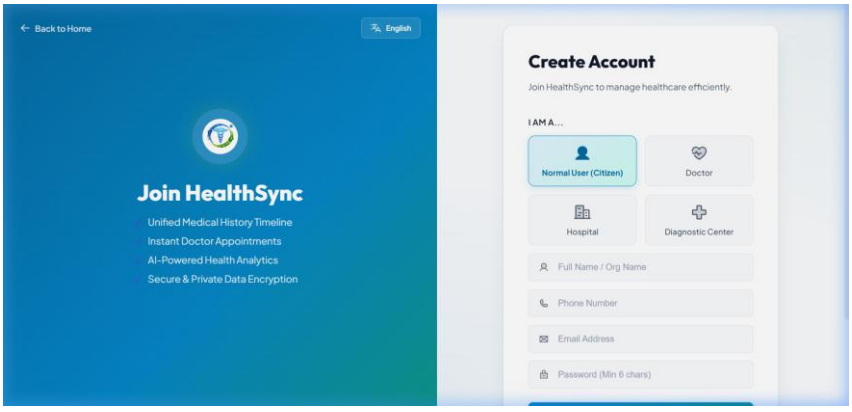


Figure 2.3: Citizen and Providers Account Registration form

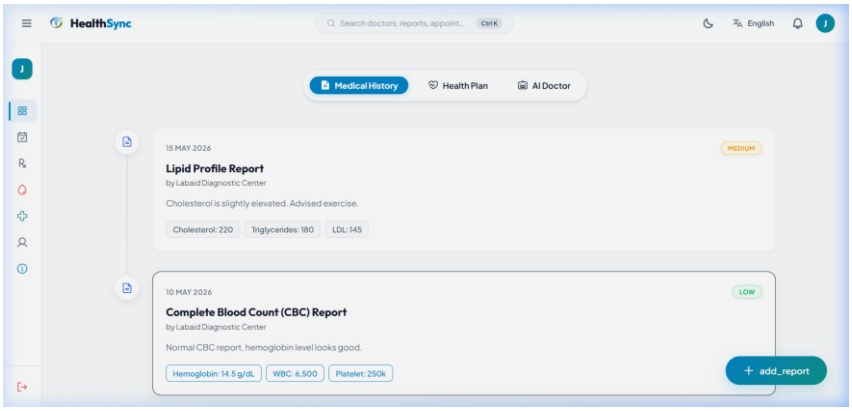


Figure 2.4: Citizen Timeline dashboard with events tracking

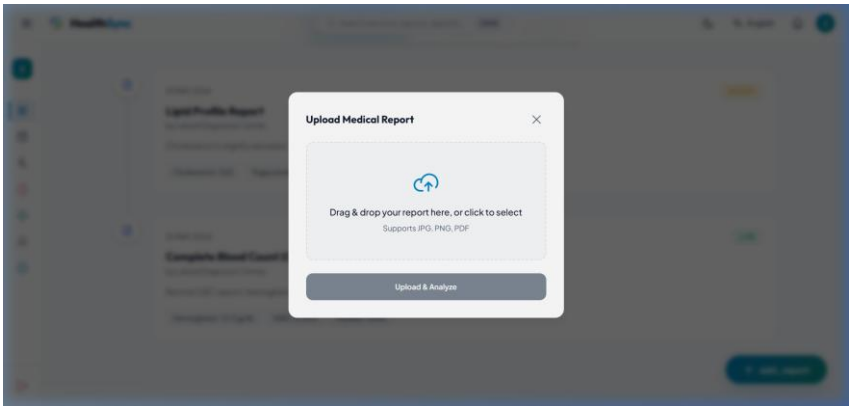


Figure 2.5: Reports Dropzone upload modal widget

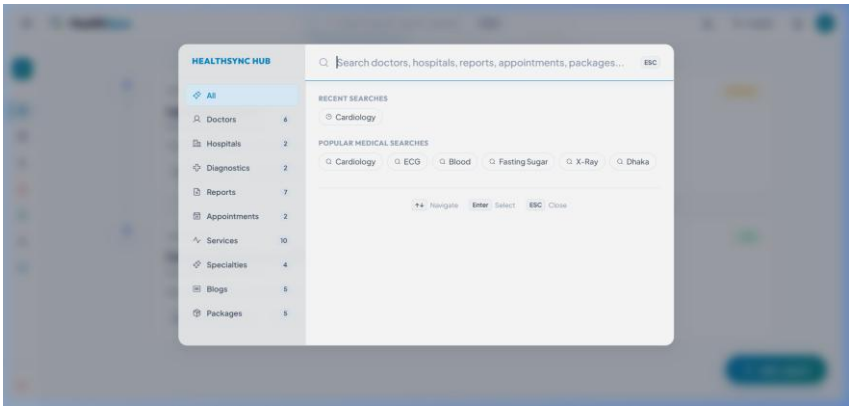


Figure 2.6: Search and explore Command Palette (Ctrl+K)

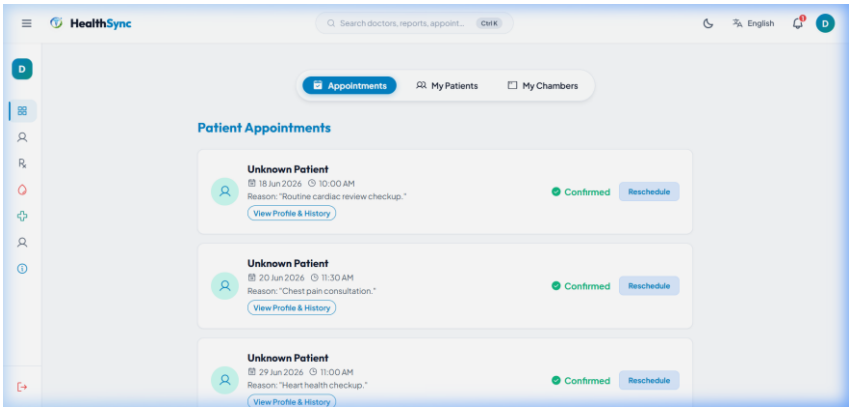


Figure 2.7: Doctor Dashboard appointments calendar view

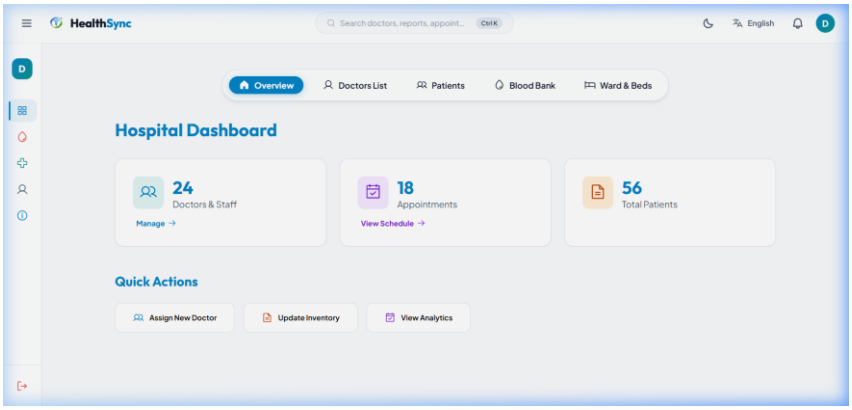


Figure 2.8: Hospital overview dashboard with bed bank tracking

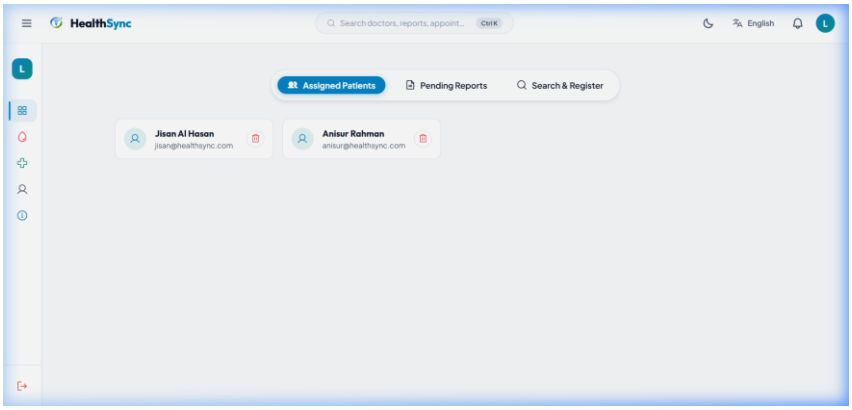


Figure 2.9: Diagnostic Center dashboard pending results queue

7. Database Design (ER Diagram)

HealthSync utilizes a normalized relational MySQL database schema. Referential integrity is maintained using foreign keys and cascade triggers.

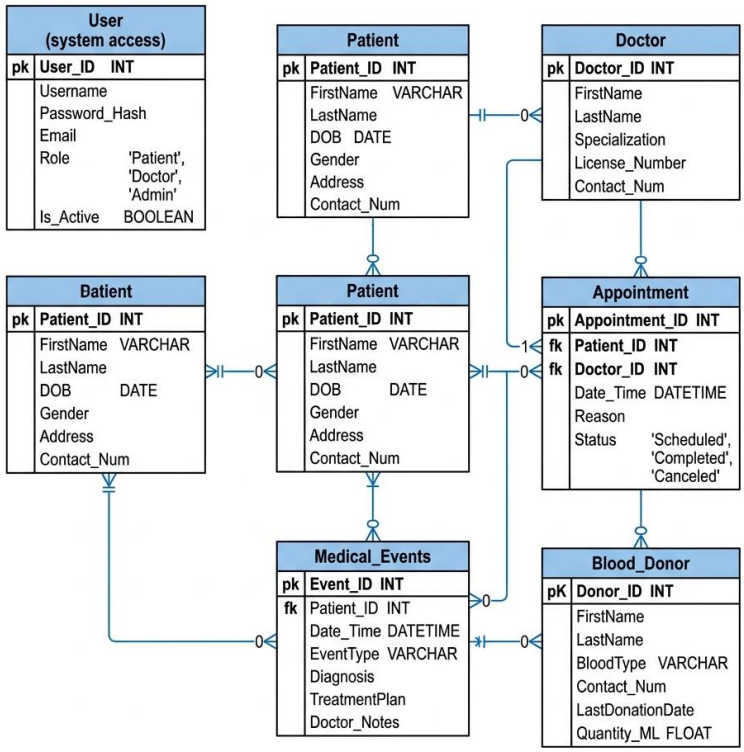


Figure 1.2: Entity-Relationship (ER) Diagram

Table 1.2: Core Database Table Metadata

Table Name	Primary Key	Description & Purpose
User	id (VARCHAR)	Stores credentials and role types (CITIZEN, DOCTOR, etc.)
blood_donors	id (INT, AI)	Tracks active blood donor availability
blood_requests	id (VARCHAR)	Tracks open blood requests, hospital location, and urgency
medical_events	id (VARCHAR)	Stores timeline records including AI OCR results and attachments
patient_payments	id (VARCHAR)	Maintains patient test orders and billing status
Appointment	id (VARCHAR)	Schedules clinical appointments

		linking Patient and Doctor
MedicalTest	id (VARCHAR)	Lookup table for standard diagnostic tests and base pricing
blood_inventory	id (INT, AI)	Tracks units of blood groups available in hospital vaults

8. Use Case Diagram

The Use Case Diagram maps interactions between system users (Citizen, Doctor, Hospital, Diagnostic) and logical features. The System Workflow Diagram demonstrates data transit paths, while the Database Relationship Diagram highlights schema foreign key connections.

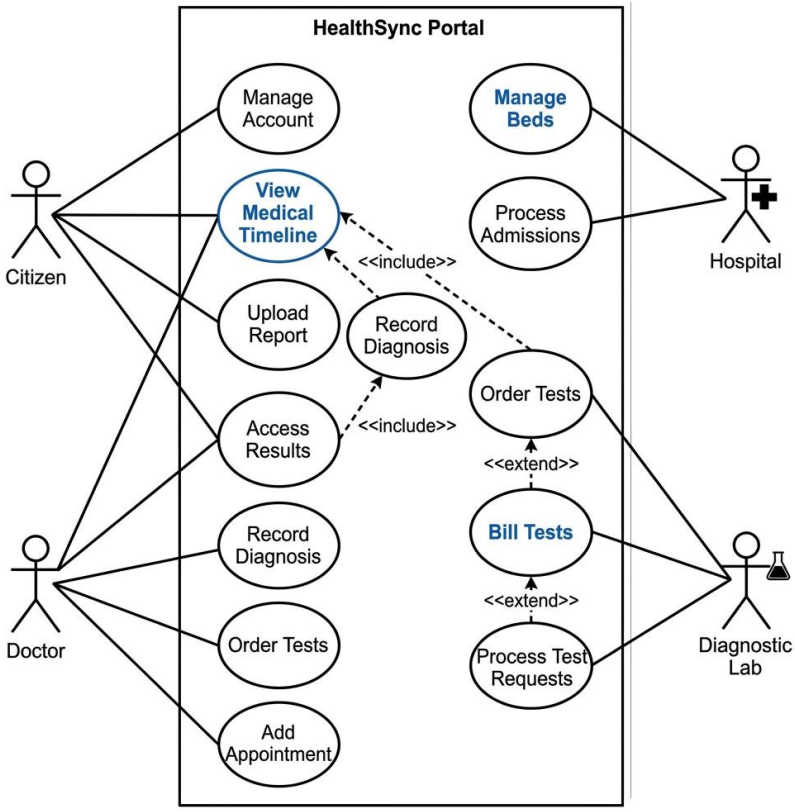


Figure 1.3: System UML Use Case Diagram

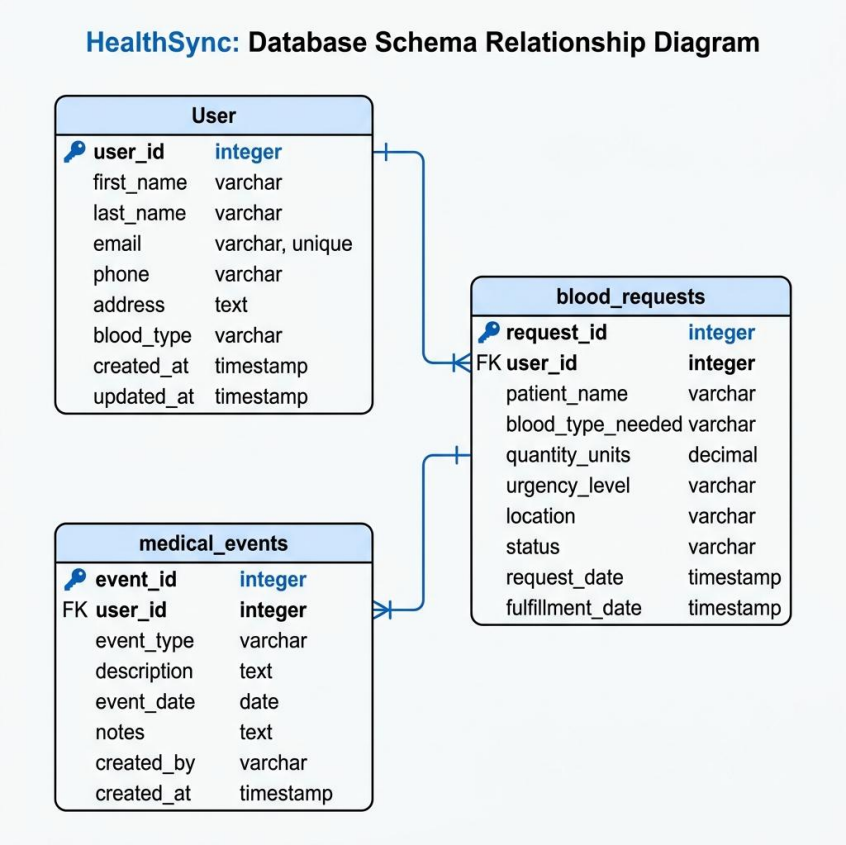


Figure 1.4: Database Schema Linkages Diagram

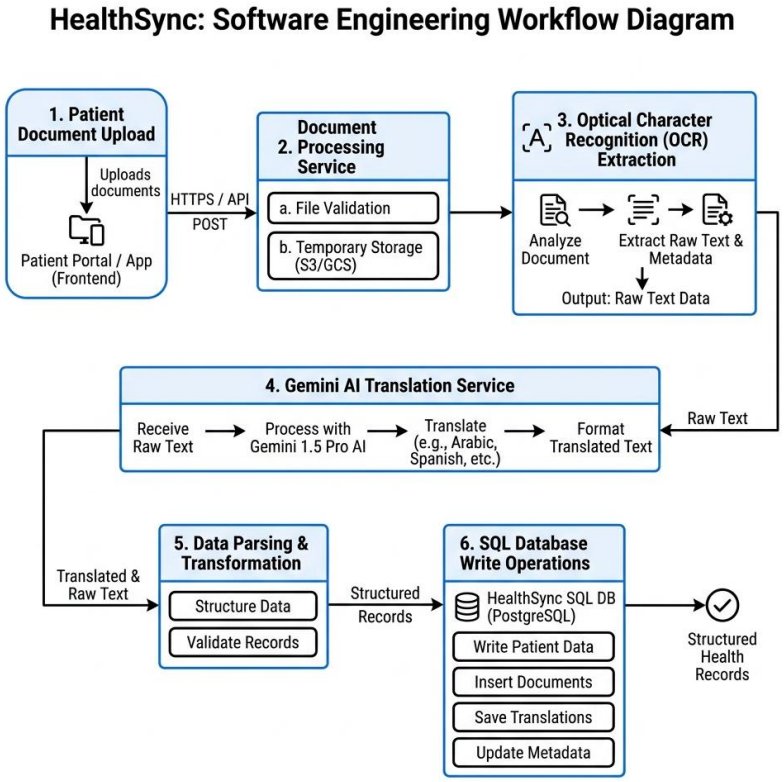


Figure 1.5: Document Processing Workflow Chart

9. Security Implementation

- Input validation: Parameterized SQL strings prevent Injection attempts; parameters are sanitized on ingress.
- Role checks: SELECT queries filter database rows based on credentials metadata, isolating user data.
- Duplicate prevention: Report uploads generate file hashes to identify duplicates, preventing database bloat.
- Environment isolation: API keys and connection passwords are isolated in environment variables (.env).
- General measures: Implementation of CORS filters and session expirations secures local storage tokens.

10. Performance Analysis

- Frontend optimization: Vite code-splitting and asset compilations reduce size.
- Database optimization: Indexing fields like User.email speeds up search operations.
- Speed metrics: pooled queries execute in less than 30ms; search triggers include debounced inputs (300ms).
- Future scaling: Redis cache layers can be used to hold static diagnostic pricing records.

11. Testing and Quality Assurance

Testing validated individual routes (unit testing), proxy queries matching Express paths (integration testing), and end-to-end user workflows (system testing). UAT reviews with students confirmed system responsiveness. Key fixes solved React asynchronous closures in search views, missing database joins in patient panels, and auto-UUID triggers in test databases.

11.6 Quality Assurance Summary

Schema validations were verified using the standalone database health diagnostic check tool, ensuring clean table links and database consistency.

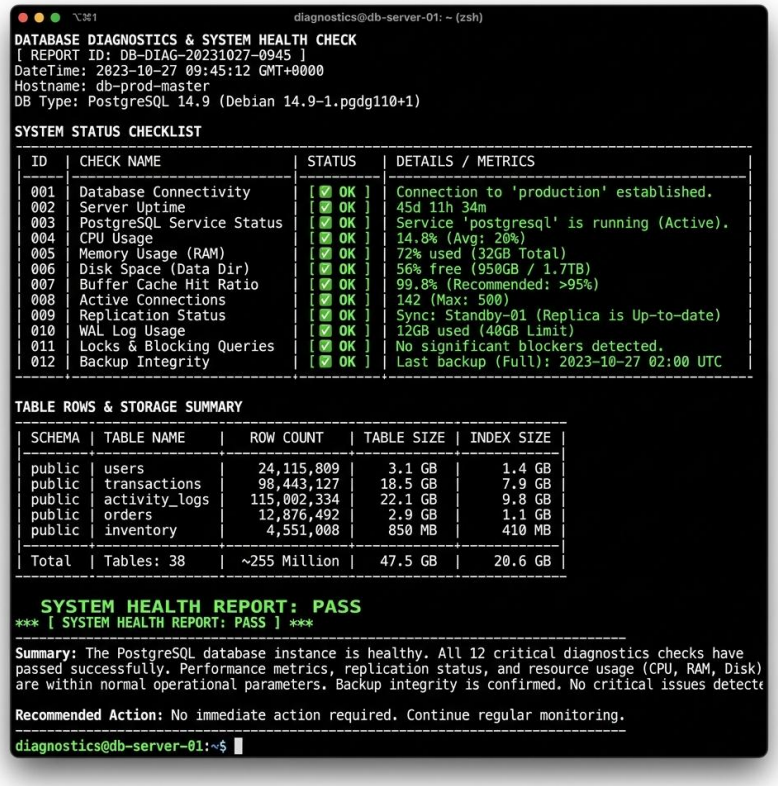


Figure 1.7: Automated Database Health Check & Diagnostics Report Console Output

12. Deployment and DevOps

For local execution, the frontend runs on port 5173 and backend Express server on port 5000. Relational schemas are initialized using schema.sql and seed configurations. Cloud deployment proposes Vercel for React SPA, Render/AWS EC2 for backend nodes, and AWS RDS for managed database services. Dev coordination uses standard Git branch merge reviews.

12.5 Continuous Improvement (DevOps Perspective)

Proposed additions include automated GitHub Actions CI pipelines to run build scripts and database validation tests on merge.

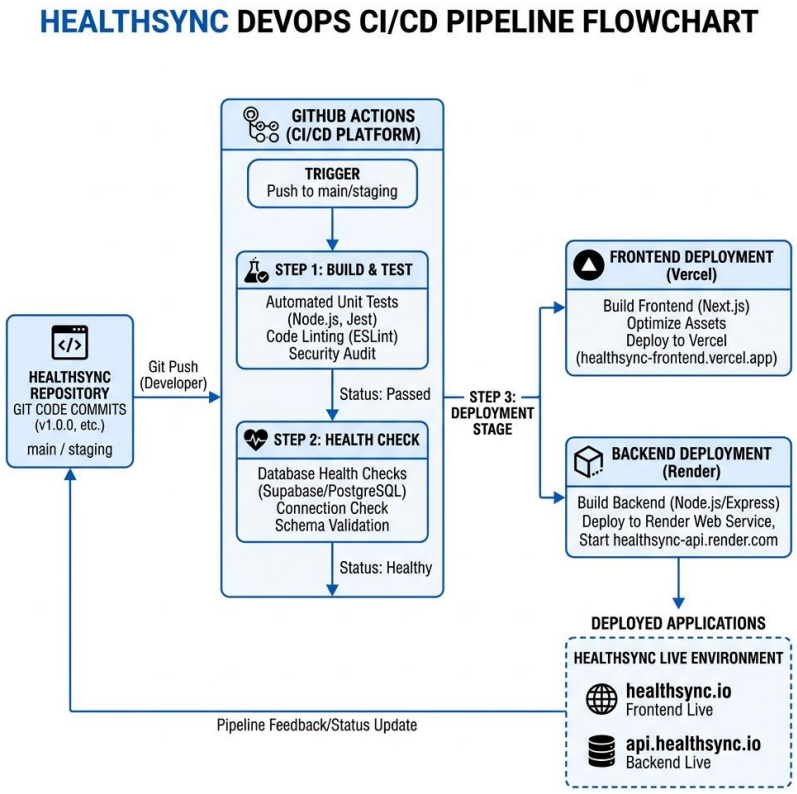


Figure 1.6: DevOps CI/CD Deployment Flow Diagram

13. Challenges and Solutions

Table 1.4: Challenges and Solutions Matrix

Challenge Encountered	Technical Solution Implemented
Missing Database Joins: Querying patient lists returned profiles as undefined, rendering patient names as 'Unknown'.	Added explicit LEFT JOIN clauses in backend select queries inside db.ts to retrieve and nest patient profiles.
Asynchronous React State Closure: Adding new users from search cards triggered search requests using stale states, producing alerts.	Refactored search functions to accept an optional emailOverride argument, bypassing stale states during user registrations.
API Column Mismatches: Billing insertions failed with SQL constraint errors due to non-existent fields like paid_amount.	Cleaned the insertion payload inside SharedTestModal.tsx to match the table column schemas defined in the MySQL database.
UUID Generation Failures: Test insertions failed due to missing default ID triggers, causing key constraint exceptions.	Extended the backend dynamic insertion router inside db.ts to identify available_tests table inputs and automatically generate UUID primary keys.

14. Future Enhancements

- Payment Integration: Deploy Stripe or SSLCommerz configurations to process patient bill settlements.
- Flutter mobile: Package web application structures into cross-platform mobile portals.
- Live chat: Run Socket.io WebSockets to connect patients directly to clinics.

15. Conclusion

HealthSync merges responsive web pages with normalized database setups and Google Gemini AI services to deliver a personal health timeline portal. The architecture balances developer convenience and database control, providing a secure and scalable foundation for clinical records digitization.

16. References

1. W3Schools - Web Tutorials (https://www.w3schools.com)
2. MySQL Reference Documentation - https://dev.mysql.com/doc/
3. MDN Web Docs - https://developer.mozilla.org/
4. Google Generative AI Developer Guides - https://ai.google.dev/gemini-api/docs

Appendix (Database SQL Schema)

The following SQL code defines the core MySQL database schema for HealthSync:

```
-- MySQL Database Schema for HealthSync (Core Tables)
CREATE DATABASE IF NOT EXISTS healthsync;
USE healthsync;

-- User table (combines auth users and public profiles)
CREATE TABLE IF NOT EXISTS `User` (
    id VARCHAR(255) PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(255) NOT NULL,
    phone VARCHAR(50),
    role VARCHAR(50) NOT NULL, -- CITIZEN, DOCTOR, HOSPITAL, DIAGNOSTIC
    blood_group VARCHAR(10),
    district VARCHAR(100),
    gender VARCHAR(20),
    specialty VARCHAR(255), -- for DOCTORs
    degree VARCHAR(255), -- for DOCTORs
    experience VARCHAR(50), -- for DOCTORs
    consultation_fee INT, -- for DOCTORs
    about TEXT, -- for DOCTORs
    chamber_address TEXT, -- for DOCTORs
    address VARCHAR(255), -- for hospitals/diagnostics/doctors
    avatar_url VARCHAR(255),
    fcm_token VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Blood Donors table
CREATE TABLE IF NOT EXISTS blood_donors (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id VARCHAR(255) UNIQUE NOT NULL,
    availability BOOLEAN DEFAULT TRUE,
    last_donation_date DATE,
    FOREIGN KEY (user_id) REFERENCES `User`(id) ON DELETE CASCADE
);

-- Blood Requests table
CREATE TABLE IF NOT EXISTS blood_requests (
    id VARCHAR(255) PRIMARY KEY,
    requester_id VARCHAR(255) NOT NULL,
    blood_group VARCHAR(10) NOT NULL,
    hospital_name VARCHAR(255) NOT NULL,
    urgency VARCHAR(20) NOT NULL, -- NORMAL, CRITICAL
    reason TEXT,
    status VARCHAR(20) DEFAULT 'OPEN', -- OPEN, FULFILLED
    accepted_count INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (requester_id) REFERENCES `User`(id) ON DELETE CASCADE
);
```